Příklady

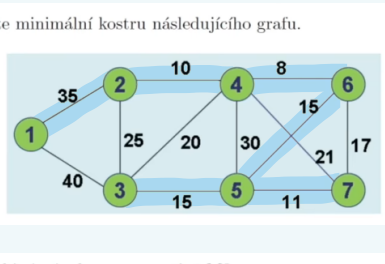
29.05.2026 12:41

1)

1. Jak je definována konvexní podmnožina lineárního prostoru?

množina C je konvexní podmnožinou lin. prostoru, pokud každá úsečka spojující lib. 2 body v C leží celá v C
neboli $\forall x_1, x_2 \in C$ a $\forall k: 0 \le k \le 1$ platí $kx_1 + (1-k)x_2 \in C$

2. Nalezněte minimální kostru následujícího grafu.



3. Co je to binární relace na množině M?

je lib. podmnožina kartézského součinu M×M

4. Stanovte optimální výrobní program firmy Microcomputer na výrobu dvou typu počítaču. Jde o počítačovou sestavu Orange se dvema disky a o počítačovou sestavu Tangerine s jedním diskem. Výrobní závod je jednak omezen počtem dostupných disků, kterých má k dispozici (20 000 kusu) a dále počtem pracovních hodin, které může uvolnit na montáž (32 000 hod/měsíc) a na kontrolu kvality (88 000 hod/měsíc). Na montáž počítače Orange je potřeba 4 hodiny, na montáž počítače Tangerine je potřeba 1 hodinu. Na kontrolu počítače Orange je potřeba 2 hodiny, na kontrolu počítače Tangerine je potřeba 7 hodin. Úkolem je navrhnout optimální výrobní program na příští měsíc maximalizující tržby firmy za předpokladu, že prodejní cena počítače Orange je 900 a počítače Tangerine 600 za kus.

	O: x_1 ↗4,33		M: x_2 ↗11,33		
disk	$2x_1$	+	$1x_2$	≤	20000 kusů
montáž	$4x_1$	+	$1x_2$	≤	32000 hod
kontrola	$2x_1$	+	$7x_2$	≤	88000 hod
	$900x_1$	+	$600x_2$	=	z

2) $A \cdot x \le b$
min/max $c^T x$
$x \in \mathbb{Z}$

$1x_1 + 2x_2 \le 3$
$4x_1 + 5x_2 \le 6$

3.3.1 Příklad ILP1a: Problém 2-rozdělení (2-Partition Problem)

Problém 2-rozdělení je klasický NP-úplný problém, který se zabývá rozdělením množiny objektů na dvě podmnožiny s co nejrovnoměrnějším součtem hodnot.

Instance: Je dána množina bankovek s počtem $n \in \mathbb{Z}^+$ a jejich hodnoty $p_1, p_2, \dots, p_n$, kde $p_i \in \mathbb{Z}^+$.

Rozhodnutí: Existuje podmnožina $S \subseteq \{1, \dots, n\}$ taková, že součet hodnot bankovek v S je roven polovině celkového součtu hodnot všech bankovek? Tj. $\sum_{i \in S} p_i = 0.5 \cdot \sum_{i=1 \dots n} p_i$?

Rozhodovací problém lze formulovat jako ILP omezení, i když ho můžeme zapsat i jinými způsoby. Zavedeme binární rozhodovací proměnné x_i, kde $x_i = 1$ pokud bankovka i patří do podmnožiny S, a $x_i = 0$ jinak.

Formulace problému 2-rozdělení jako ILP:

$$\begin{aligned} &\text{min} && 0 \\ &\text{za podmínek} && \sum_{i=1 \dots n} x_i \cdot p_i = 0.5 \cdot \sum_{i=1 \dots n} p_i \\ &\text{parametry:} && n \in \mathbb{Z}^+, p_i \in \mathbb{Z}^+ \\ &\text{proměnné:} && x_i \in \{0,1\} \quad \text{pro } i = 1 \dots n \end{aligned}$$

Tato formulace se snaží minimalizovat nulovou účelovou funkci (protože hledáme pouze existence řešení, ne optimalizaci hodnoty) s jediným klíčovým omezením, že součet hodnot vybraných bankovek musí být přesně polovina celkového součtu. Proměnné x_i jsou binární, což zajišťuje, že každá bankovka je buď vybrána, nebo ne.

- x_i: Binární proměnná, která je 1, pokud bankovka i patří do podmnožiny S, jinak 0.
- p_i: Hodnota i-té bankovky.
- n: Celkový počet bankovek.

Příklad: two partition

1000 1000 500 200 200 200

$x_i = 1$ iff $i \in S$ — n_i hodnota i-té bankovky
$x_i = 0$ není v S

existuje? jen rozhodujeme → min 0 (konstanta)

podm. součet je roven polovině hodnota → $\sum_{i=1..n} x_i \cdot n_i = 0{,}5 \cdot \sum_{i=1..n} n_i$

parametry $m \in \mathbb{Z}^+$, $n_i \in \mathbb{Z}^+$

proměnné $x_i \in \{0,1\}$ pro $i = 1, \dots m$

frakcionální $x_i \in \langle 0,1 \rangle$ můžeme dělit $x_i \le 1$

optimalizační → minimalizovat rozdíl
C_1 - hodnota skupiny 1, C_2 - 2 } C_{max} → větší z nich

min C_{max}
$\sum x_i \cdot n_i \le C_{max}$
$\sum (1-x_i)\, n_i \le C_{max}$
$m \in \mathbb{Z}^+$
$n_i \in \mathbb{Z}^+$
$x \in \{0,1\}$
$C_{max} \in \mathbb{R}^+$

3.3.4 Příklad ILP2a: Nejkratší cesty v grafu

Problém nalezení nejkratší cesty v orientovaném grafu je základní úlohou v teorii grafů a lze jej formulovat jako problém lineárního programování (LP), pokud hledáme cestu s pevnou délkou nebo minimalizujeme celkové náklady.

Instance: Je dán orientovaný graf G s n uzly a matice vzdáleností (nebo nákladů) $c: V \times V \to \mathbb{R}^+$, kde c_{ij} je vzdálenost z uzlu i do uzlu j. Jsou dány dva specifické uzly: počáteční uzel v_1 a koncový uzel v_n.

Cíl: Najít nejkratší cestu z uzlu v_1 do uzlu v_n, nebo rozhodnout, zda je v_n z v_1 nedosažitelný.

LP formulace tohoto problému může být vytvořena pomocí fyzikální analogie, kde:

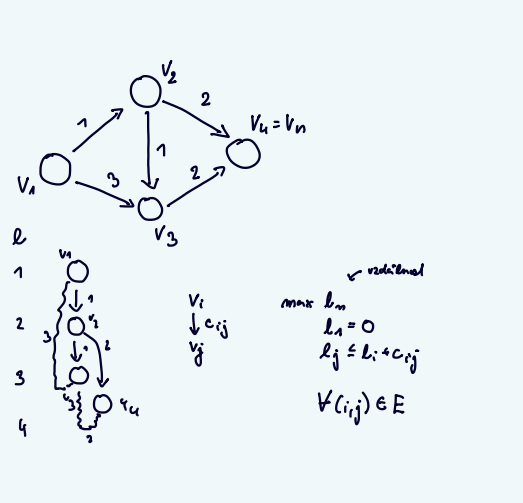
- **Uzel** = kulička
- **Hrana** = struna (uvažujeme matici symetrických vzdáleností $c_{ij} = c_{ji}$)
- Uzel v_1 je **pevně ukotven**, zatímco ostatní uzly jsou **taženy gravitací**, **napnutá struna** = nejkratší cesta.

Formulace jako LP (pro nalezení nejdelší cesty mezi dvěma uzly, což lze s malou úpravou využít pro nejkratší cesty):

$$\begin{aligned} &\text{max} && l_n \\ &\text{za podmínek} && l_1 = 0 \\ & && l_j \le l_i + c_{ij} \quad \text{pro } i = 1 \dots n, j = 1 \dots n, i \ne j \\ &\text{parametry:} && n \in \mathbb{Z}^+, c_{ij} \in \mathbb{R}^+ \\ &\text{proměnné:} && l_i \in \mathbb{R} \quad \text{pro } i = 1 \dots n \end{aligned}$$

Tato formulace s maximalizací l_n je typická pro nalezení nejdelší cesty v acyklickém grafu nebo pro Bellmanovu-Fordovu rovnici v modifikované podobě. Proměnná l_i zde reprezentuje délku nejkratší cesty z počátečního uzlu v_1 do uzlu v_i. Omezení $l_j \le l_i + c_{ij}$ zajišťuje, že délka cesty do j nemůže být větší než délka cesty do i plus délka hrany c_{ij}.

- l_i: Reálná proměnná reprezentující délku nejkratší cesty z v_1 do uzlu i.
- c_{ij}: Náklady (vzdálenost) hrany z uzlu i do uzlu j.
- n: Celkový počet uzlů v grafu.



$v_i \downarrow c_{ij}\ v_j$

max l_n ← vzdálenost
$l_1 = 0$
$l_j \le l_i + c_{ij}$
$\forall (i,j) \in E$

3.3.5 Příklad ILP3: Problém obchodního cestujícího (Traveling Salesman Problem, TSP)

Asymetrický problém obchodního cestujícího (Asymmetric Traveling Salesman Problem, ATSP) je klasický NP-těžký problém kombinatorické optimalizace.

Instance: Je dán úplný orientovaný graf K_n (kde mezi každou dvojicí uzlů existují hrany v obou směrech) a matice vzdáleností $c: V \times V \to \mathbb{Q}^+$ (kladná racionální čísla), kde c_{ij} je vzdálenost z uzlu i do uzlu j.

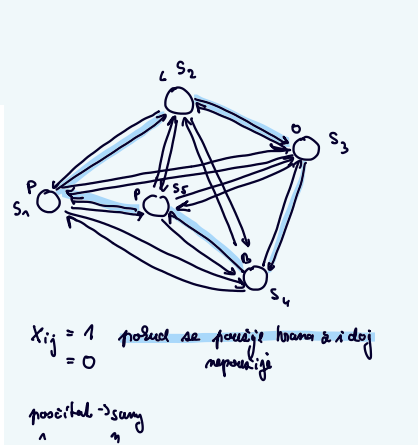
Cíl: Najít nejkratší Hamiltonovský cyklus (neboli cestu, která navštíví všechny uzly grafu právě jednou a vrátí se do počátečního uzlu). Jedná se o podgraf s n uzly a n hranami takový, že posloupnost uzlů a hran $v_{[1]}, e_{[1]}, \dots, v_{[n]}, e_{[n]}, v_{[1]}$ je uzavřený orientovaný tah, kde $v_i \ne v_j$ pro $1 \le i < j \le n$.

Problém ATSP je formulován pomocí binárních rozhodovacích proměnných x_{ij}, kde: $x_{ij} = 1$ právě tehdy, když uzel i v cyklu bezprostředně předchází uzlu j, jinak $x_{ij} = 0$.

Tradiční omezení pro vstup a výstup z uzlů (např. do každého uzlu se vstoupí právě jednou a z každého uzlu se vystoupí právě jednou) nepostačují k plné formulaci TSP, protože by je splnily i libovolné **disjunktní sub-cykly** (tj. řešení skládající se z několika menších, oddělených cyklů namísto jednoho velkého cyklu procházejícího všemi uzly). K eliminaci těchto sub-cyklů používáme proměnné s_i, které reprezentují "čas" vstupu do uzlu i.

Formulace ATSP s eliminací sub-cyklů:

$$\begin{aligned} &\text{min} && \sum_{i=1 \dots n} \sum_{j=1 \dots n, j \ne i} c_{ij} \cdot x_{ij} \\ &\text{za podmínek} && x_{ii} = 0 \quad \text{pro } i = 1 \dots n \quad \text{(zabraňuje smyčkám)} \\ & && \sum_{j=1 \dots n, j \ne i} x_{ij} = 1 \quad \text{pro } i = 1 \dots n \quad \text{(z každého uzlu se vystupuje právě jednou)} \\ & && \sum_{i=1 \dots n, i \ne j} x_{ij} = 1 \quad \text{pro } j = 1 \dots n \quad \text{(do každého uzlu se vstupuje právě jednou)} \\ & && s_i + c_{ij} - (1 - x_{ij}) \cdot M \le s_j \quad \text{pro } i = 1 \dots n, j = 2 \dots n, i \ne j \quad \text{(eliminace sub-cyklů)} \\ &\text{parametry:} && M \in \mathbb{Z}^+, n \in \mathbb{Z}^+, c_{ij} \in \mathbb{Q}^+ \\ &\text{proměnné:} && x_{ij} \in \{0,1\} \quad \text{pro } i,j = 1 \dots n \\ & && s_i \in \mathbb{R}_0^+ \quad \text{pro } i = 1 \dots n \end{aligned}$$



$x_{ij} = 1$ pokud se použije hrana z i do j
$= 0$ nepoužije

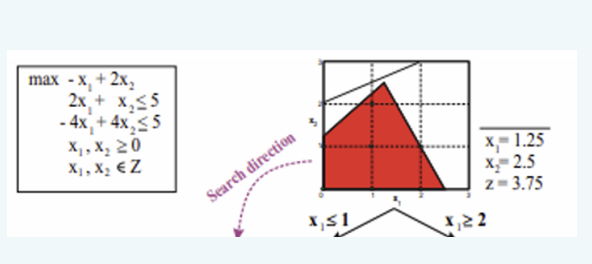
pocital → smyčky

1 vstupní, 1 výstupní pro každé město

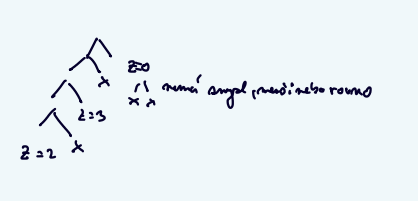
Ale → může vzniknout sub-cykly → chceme 1
→ součet hran s_i
pro c_{ij}: $s_j \ge s_i + c_{ij}$ chceme nepovolit pro

vynásobím nějakým velkým číslem $M \cdot (1 - x_{ij}) + s_j \ge s_i + c_{ij}$
↳ vždy pravda → rovnice zmizí
v závislosti na proměnné x_{ij} (zapnutí/vypnutí)
když $x = 1$ (bude hrana) → $M \cdot 0$, rovnice zapnutá

nerozdělitelnost cyklu (eliminace hran vstupujících do poč. vrcholu)
nevstup do vrcholu $j = 1$ → $j \in 2, \dots m$



max $-x_1 + 2x_2$
$2x_1 + x_2 \le 5$
$-4x_1 + 4x_2 \le 5$
$x_1, x_2 \ge 0$
$x_1, x_2 \in \mathbb{Z}$



3.7 Speciální případy ILP - Příklad ILP2b: Nejkratší cesty (opakování)

Znovu se podíváme na problém nejkratších cest, tentokrát s důrazem na to, jak může být formulován tak, aby přirozeně vedl k celočíselnému řešení, i když se zdá, že jde o LP.

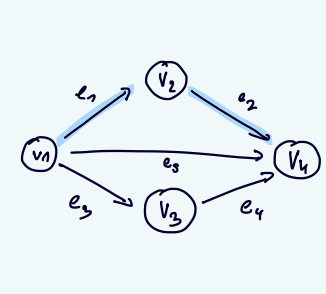
Nejkratší cesta v digrafu:

Instance: Je dán orientovaný graf G reprezentovaný maticí incidence $W: V \times E \to \{-1, 0, +1\}$. Element $w_{ij} = +1$ pokud hrana j vychází z uzlu i, a $w_{kj} = -1$ pokud hrana j vstupuje do uzlu k. Dále je dán vektor vzdáleností (nákladů) hran $c \in \mathbb{R}^+$, kde c_j je vzdálenost hrany j. Jsou dány dva uzly, počáteční uzel s a koncový uzel $t \in V$.

Cíl: Najít nejkratší cestu z s do t, nebo rozhodnout, že t je z s nedosažitelný.

LP formulace problému nejkratších cest:

$$\begin{aligned} &\text{min} && \sum_{j=1 \dots m} c_j \cdot x_j \\ &\text{za podmínek} && \sum_{j=1 \dots m} w_{sj} \cdot x_j = 1 \quad \text{(zdroj s)} \\ & && \sum_{j=1 \dots m} w_{tj} \cdot x_j = -1 \quad \text{(cíl t)} \\ & && \sum_{j=1 \dots m} w_{ij} \cdot x_j = 0 \quad \text{pro } i \in V \setminus \{s,t\} \quad \text{(ostatní uzly)} \\ &\text{parametry:} && W \in \{-1, 0, 1\}^{n \times m}, c_j \in \mathbb{R}^+ \\ &\text{proměnné:} && x_j \in \mathbb{R}_0^+ \quad \text{pro } j = 1 \dots m \end{aligned}$$



	e_1	e_2	e_3	e_4	e_5
V_1	+1	0	+1	0	+1
V_2	−1	+1	0	0	0
V_3	0	0	−1	+1	0
V_4	0	−1	0	−1	−1

použita e_2 e_5 nepoužita
start $= 1 = 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1$

$\sum_j w_{ij} \cdot x_j = 1$ start, $= 0$ else, $= -1$ end
$w_{sj}\, x_j$
$w_{ej}\, x_j$

$x_j = 1$ hrana j součástí cesty (0 - nevede)

Příklad na vlastnost po sobě jdoucích jedniček: Rozvrhování pracovních sil

Minimalizujte celkové náklady na zaměstnance pro čtyřhodinové bloky.

Požadavky a omezení:

- **Minimální personální požadavky:** Ráno - 4 zaměstnanci, Odpoledne - 7, Večer - 5, Noc - 2.
- **Pokryté bloky směnami:** (1) Ráno+Odpoledne, (2) Odpoledne+Večer, (3) Večer+Noc, (4) Noc, (5) Celý den.
- **Náklady na zaměstnance:** Jeden zaměstnanec na směně j stojí c_j.

Nechť x_j je počet zaměstnanců přiřazených směně j.

Formulace ILP:

min $\sum_{j}^{5} x_j \cdot c_j$

$1x_1 + 0x_2 + 0x_3 + 0x_4 + 1x_5$	≥	4	ráno
$1x_1 + 1$	1	0	0	1	≥ 7	odpoledne
$0x_1 + 1$	1	0	1	≥ 5		večer
$0x_1 + 0$	1	1	1	≥ 2		noc

$x_j \in \mathbb{Z}^+$
$c_j \in \mathbb{R}^+$

po sobě jdoucí jedničky TUM - totálně unimodulární → determinant každé □ submatice $= -1, 0, 1$
matice omezení → stačí LP relaxace
→ pokud jsou LP může být neceločíselné → NP těžký (B-B)

budova i	1	2	3	4	5	6
cena [mil. CZK]	5	7	4	3	4	6
příjem [tisíc CZK]	16	22	12	8	11	19

Tabulka 3.1: Data o budovách pro investiční problém.

Cíl: Maximalizovat celkový příjem z pronájmu vybraných budov.

Omezení:

- **Investiční rozpočet:** Celková částka, kterou lze investovat, je 14 milionů CZK.
- **Jednorázový nákup:** Každou budovu lze koupit pouze jednou (buď ji koupíme, nebo nekoupíme).

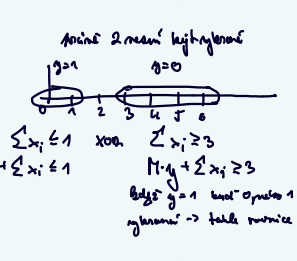
$x_i = 1$ budova i koupena

max $16x_1 + 22x_2 + 12x_3 + 8x_4 + 11x_5 + 19x_6$
$5x_1 + 7x_2 + 4x_3 + 3x_4 + 4x_5 + 6x_6 \le 14$
$x_i \in \{0,1\}$ $i = 1, 2 \dots 6$

implikace $x_1 \Rightarrow x_2$ pokud x_1 tak i x_2 | $x_1 \le x_2$
$x_3 \Rightarrow \neg x_4$ pokud x_3 tak ne x_4 | $x_3 + x_4 \le 1$
exkl. disj. x_5 XOR x_6 buď x_5 nebo x_6 nebo obě | $x_5 + x_6 = 1$

x_1 musí a x_2 nesmí | $x_1 = 1$, $x_2 = 0$
musí alespoň 3 | $\sum x_i \ge 3$
$(x_1 \text{ AND } x_2) \Rightarrow x_3$ | $x_1 + x_2 - x_3 \le 1$

právě 2 nesmí být splněné



$\sum x_i \le 1$ xor $\sum x_i \ge 3$
$M(1-y) + \sum x_i \le 1$ $M \cdot y + \sum x_i \ge 3$
když $y = 1$ tak první splněn
vynucení → tato rovnice se vypne

Uvažujme problém výrobce oděvů, který produkuje T-shirty, košile (shirt) a róby (robe). Cílem je maximalizovat zisk z výroby s ohledem na omezené zdroje práce a materiálu.

produkt	T-shirt	shirt	robe	capacity	unit cost[CZK]
labor [person-hour]	3	2	6	160	1000
material [meter]	4	3	4	130	200
income [1000 CZK]	9.8	6.6	13.8		

Cíl: Maximalizovat celkový zisk, který je definován jako celkový příjem minus celkové náklady (výdaje).

Omezení:

- **Kapacita práce:** Celkový počet dostupných osobohodin je maximálně 160.
- **Kapacita materiálu:** Celkové množství dostupného materiálu je maximálně 130 metrů.

x_i množství oblečení i (triko, ko, rob)

	příjem	výdej osoby	výdej materiál	zisk
Triko	9800 Kč	− 3000 Kč	− 4·200 = 800 Kč	6000
košile	6600 Kč	2000	600 Kč	4000
róba	13800 Kč	6000	800 Kč	7000

max $6000x_1 + 4000x_2 + 7000x_3$
$3x_1 + 2x_2 + 6x_3 \le 160$
$4x_1 + 3x_2 + 4x_3 \le 130$
$x_{1,2,3} \in \mathbb{Z}_0^+$

Rozšíříme problém o dodatečné omezení: **Musí být pokryty fixní náklady na pronájem stroje.** Pro výrobu každého produktu je nutné pronajmout specifický stroj s fixními náklady, pokud se daný produkt vůbec vyrábí.

produkt	T-shirt	shirt	robe
machine cost [CZK]	200 000	150 000	100 000

Zavedeme **binární proměnné** y_i, kde: $y_i = 1$ pokud je produkt i vyráběn ($x_i \ge 1$), a $y_i = 0$ pokud produkt i není vyráběn ($x_i = 0$).

y_i pokud se i vyrábí ($x_i \ge 1$)
$y_i = 0$ pokud $x_i = 0$

max $6000x_1 + 4000x_2 + 7000x_3 - 200000y_1 - 150000y_2 - 100000y_3$
$3x_1 + 2x_2 + 6x_3 \le 160$
$4x_1 + 3x_2 + 4x_3 \le 130$
$x_1 \le M y_1$
$x_2 \le M y_2$
$x_3 \le M y_3$
$x_{123} \in \mathbb{Z}_0^+$
$y_{123} \in \{0,1\}$

MUSÍME PROPOJIT

x	y	
0	0	nevyrábí
0	1	✓ (nesmí)
≥1	0	nelze
≥1	1	✓

x vyrábí ≤ velké·1 ✓
0 nevyrábí ≤ velké·1 ✓ (dražší)
x vyrábí ≤ velké·0 ✗ nelze
0 nevyrábí ≤ velké·0 ✓ =

Další omezení se týká celkové dostupné kapacity práce, která může být pružná: **Celkové množství práce za týden může být maximálně 40, 80, 120 nebo 160 hodin**, v závislosti na počtu zaměstnanců na plný úvazek (FTEs). Adaptace kritéria: **Náklady na práci jsou proporcionální počtu FTEs, přičemž jeden FTE stojí 40 CZK.** Tedy chceme, aby celková spotřeba práce $3x_1 + 2x_2 + 6x_3$ byla ≤ 40, nebo ≤ 80, ≤ 120, nebo ≤ 160.

úrovně 40, 80, 120 nebo 160
$3x_1 + 2x_2 + 6x_3 \le 40v_1 + 80v_2 + 120v_3 + 160v_4$
$\sum v_i = 1$ — pouze 1 úroveň
↳ úrovně do ceny Tsh 9800 − $40v_1 - 80v_2 - 120v_3 - 160v_4$
Sh 6000
rob 13000
$v_i \in \{0,1\}$

Při modelování problémů pomocí ILP se často potřebujeme vyjádřit, že musí platit buď první, druhé, nebo obě omezení. Předpokládejme, že máme dvě omezení, například:

$$2x_1 + 2x_2 \le 8$$
$$2x_3 - 2x_4 \le 2$$

a chceme, aby platilo alespoň jedno z nich.

$2x_1 + 2x_2 \le 8 + My$
$2x_3 - 2x_4 \le 2 + M(1-y)$

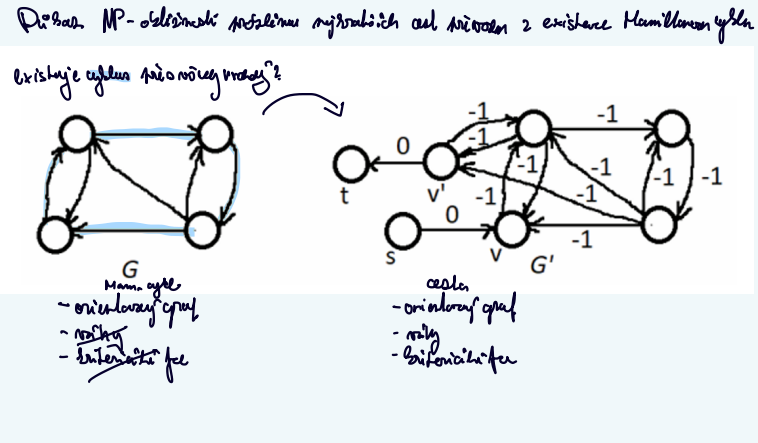
Generalizujeme předchozí problém: Máme N omezení a požadujeme, aby platilo **alespoň K z nich**. Omezení jsou typu:

$$f_1(x_1, x_2, \dots, x_n) \le b_1$$
$$f_2(x_1, x_2, \dots, x_n) \le b_2$$
$$\vdots$$
$$f_N(x_1, x_2, \dots, x_n) \le b_N$$

$f_1(\ ) \le b_1 + M \cdot y_1$
$f_2(\ ) \le b_2 + M \cdot y_2$
⋮
$f_m(\ ) \le b_m + M \cdot y_m$
alespoň k platí $\sum y_i \le m - k$
kde $y_i = 0$

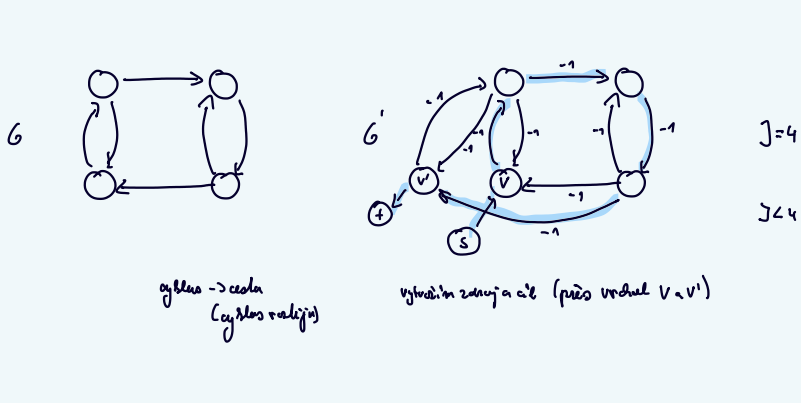
Důkaz NP-obtížnosti problému nejkratších cest převodem z existence Hamiltonova cyklu

Existuje cyklus přes všechny vrcholy?



Ham. cykly
- orientovaný graf
- ~~váhy~~
- ~~kriteriální~~ fce

cesta
- orientovaný graf
- váhy
- kriteriální fce



cyklus → cesta (cyklus rozdělíme)

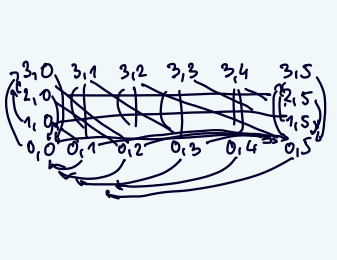
vytvoříme zdroj a cíl (přes vrchol v a v')

J=4 k vrcholy v původním G, 5 vrcholů v G'

J<4 nepodařilo přes všechny vrcholy
→ Ham. cyklus neexistuje

4.12.1 Příklad: Měření hladiny vody (Water Level Measurement)

Jste na břehu jezera a máte k dispozici prázdnou 3litrovou lahev a prázdnou 5litrovou lahev. Vaším úkolem je získat přesně 4 litry vody ve větší (5litrové) lahvi. Nemáte k dispozici žádné další vybavení pro měření hladiny vody. Tento klasický problém lze reprezentovat grafem a řešit pomocí algoritmu nejkratší cesty. Uzly grafu by představovaly možné stavy (množství vody v obou lahvích, např. (x,y) kde x je voda v 3l lahvi a y je voda v 5l lahvi), a hrany by reprezentovaly povolené operace (např. naplnit lahev, vylít lahev, přelít z jedné do druhé).



Výroba hraček

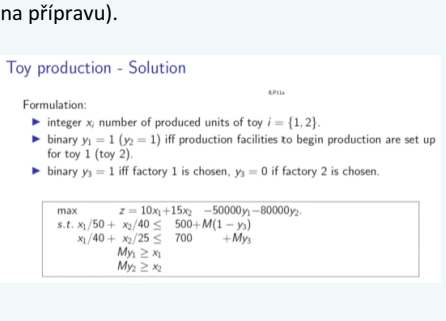
Společnost plánuje výrobu dvou druhů hraček (Hračka 1 a Hračka 2). Za každý prodaný kus Hračky 1 má společnost čistý zisk 10 peněžních jednotek, za každý kus Hračky 2 je zisk 15 jednotek.

Příprava výroby každé hračky si však žádá jednorázové fixní náklady (např. na zakoupení forem nebo nastavení linek), bez ohledu na to, kolik kusů se nakonec vyrobí. Pokud se společnost rozhodne vyrábět Hračku 1, musí jednorázově zaplatit 50 000. Pokud se rozhodne vyrábět Hračku 2, musí jednorázově zaplatit 80 000.

Společnost má k dispozici **dvě různé továrny**, ale z logistických důvodů musí celou produkci umístit **pouze do jedné z nich**. Továrny se liší svou celkovou kapacitou i rychlostí výroby:

- **Továrna 1:** Má k dispozici celkem **500 hodin** strojového času. Hračku 1 dokáže vyrábět rychlostí 50 kusů za hodinu, Hračku 2 rychlostí 40 kusů za hodinu.
- **Továrna 2:** Má k dispozici celkem **700 hodin** strojového času. Hračku 1 dokáže vyrábět rychlostí 40 kusů za hodinu, Hračku 2 rychlostí 25 kusů za hodinu.

Cíl: Formulujte model celočíselného lineárního programování, který určí, jakou továrnu vybrat a kolik kusů jednotlivých hraček vyrobit, aby byl maximalizován celkový zisk (celkové příjmy minus fixní náklady na přípravu).



Toy production - Solution

Formulation:
- ▶ integer x_i number of produced units of toy $i \in \{1,2\}$.
- ▶ binary $y_1 = 1$ ($y_2 = 1$) iff production facilities to begin production are set up for toy 1 (toy 2).
- ▶ binary $y_3 = 1$ iff factory 1 is chosen, $y_3 = 0$ if factory 2 is chosen.

$$\begin{aligned} \max \quad & z = 10x_1 + 15x_2 - 50000y_1 - 80000y_2 \\ \text{s.t.} \quad & x_1/50 + x_2/40 \le 500 + M(1-y_3) \\ & x_1/40 + x_2/25 \le 700 + My_3 \\ & My_1 \ge x_1 \\ & My_2 \ge x_2 \end{aligned}$$

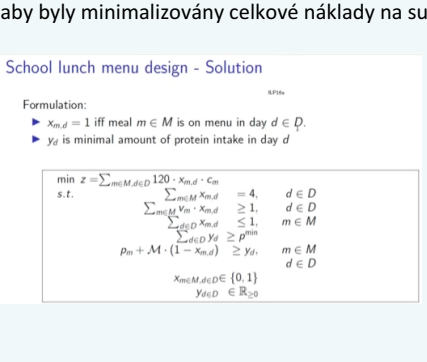
Návrh školního jídelníčku

Školní jídelna sestavuje menu na dané období (množina dnů D) z dostupné databáze receptů (množina jídel M). Pro každé jídlo $m \in M$ jsou známy náklady na jednu porci c_m, obsah bílkovin p_m a binární indikátor v_m, který určuje, zda je jídlo vegetariánské ($v_m = 1$) či nikoliv ($v_m = 0$). Jídelna denně vaří pro 120 strávníků.

Při sestavování jídelníčku musí být dodržena následující provozní a nutriční pravidla:

1. Každý den musí být v nabídce na výběr přesně 4 různá jídla.
2. Každý den musí být mezi nabízenými jídly alespoň jedno vegetariánské.
3. Z důvodu pestrosti stravy nesmí být žádné jídlo servírováno více než jednou za celé sledované období.
4. Jídelna musí garantovat určitý nutriční standard. Předpokládejme nejhorší možný scénář: strávník si každý den vybere z nabídky 4 jídel to, které má absolutně *nejmenší* obsah bílkovin. I v tomto nejhorším případě musí součet bílkovin přijatých tímto strávníkem za celé období dosáhnout alespoň minimální požadované hodnoty $p^{\min}$

Cíl: Formulujte model celočíselného lineárního programování, který vybere jídla pro jednotlivé dny tak, aby byly minimalizovány celkové náklady na suroviny pro všechny strávníky za celé období.



School lunch menu design - Solution

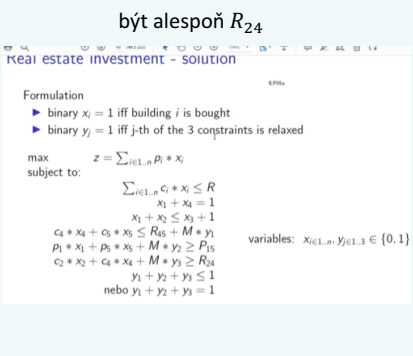
Formulation:
- ▶ $x_{m,d} = 1$ iff meal $m \in M$ is on menu in day $d \in D$.
- ▶ y_d is minimal amount of protein intake in day d

$$\begin{aligned} \min \quad & z = \textstyle\sum_{m \in M, d \in D} 120 \cdot x_{m,d} \cdot c_m \\ \text{s.t.} \quad & \textstyle\sum_{m \in M} x_{m,d} = 4, && d \in D \\ & \textstyle\sum_{m \in M} v_m \cdot x_{m,d} \ge 1, && d \in D \\ & \textstyle\sum_{d \in D} x_{m,d} \le 1, && m \in M \\ & \textstyle\sum_{d \in D} y_d \ge p^{\min} \\ & p_m + M \cdot (1 - x_{m,d}) \ge y_d, && m \in M, d \in D \\ & x_{m \in M, d \in D} \in \{0,1\} \\ & y_{d \in D} \in \mathbb{R}_{\ge 0} \end{aligned}$$

Investice do nemovitostí

Investor má na výběr z n budov ke koupi. Každá budova i má svou pořizovací cenu c_i a očekávaný zisk (nebo výnos) p_i. Investor disponuje celkovým rozpočtem R. Cílem je vybrat takovou podmnožinu budov, která maximalizuje celkový zisk, a přitom dodržuje následující strategická pravidla:

1. **Rozpočet:** Celková cena zakoupených budov nesmí překročit rozpočet R
2. **Exkluzivita:** Investor musí koupit *právě jednu* z budov 1 a 4 (nesmí koupit obě a nesmí je obě odmítnout).
3. **Podmíněná investice:** Pokud investor koupí budovu 1 *a zároveň* budovu 2, pak musí povinně koupit i budovu 3.
4. **Volitelná portfoliová pravidla:** Existují tři specifická pravidla pro složení portfolia. Investor musí splnit **alespoň dvě** (případně *přesně dvě*, viz vysvětlení níže) z těchto tří pravidel:
    - ○ Pravidlo A: Společné pořizovací náklady na budovy 4 a 5 nesmí překročit dílčí rozpočet R_{45}
    - ○ Pravidlo B: Společný zisk z budov 1 a 5 musí být minimálně P_{15}
    - ○ Pravidlo C: Společné náklady (nebo jiná metrika určená koeficienty c) na budovy 2 a 4 musí být alespoň R_{24}



Real estate investment - solution

Formulation
- ▶ binary $x_i = 1$ iff building i is bought
- ▶ binary $y_j = 1$ iff j-th of the 3 constraints is relaxed

$$\begin{aligned} \max \quad & z = \textstyle\sum_{i \in 1..5} p_i * x_i \\ \text{subject to:} \quad & \textstyle\sum_{i \in 1..5} c_i * x_i \le R \\ & x_1 + x_4 = 1 \\ & x_1 + x_2 \le x_3 + 1 \\ & c_4 * x_4 + c_5 * x_5 \le R_{45} + M * y_1 \\ & p_1 * x_1 + p_5 * x_5 + M * y_2 \ge P_{15} \\ & c_2 * x_2 + c_4 * x_4 + M * y_3 \ge R_{24} \\ & y_1 + y_2 + y_3 \le 1 \\ & \text{nebo } y_1 + y_2 + y_3 = 1 \end{aligned}$$

variables: $x_{i \in 1..5}, y_{i \in 1..3} \in \{0,1\}$

Investice do akcií

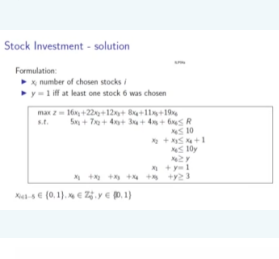
Investor sestavuje portfolio a má na výběr z 6 různých investičních příležitostí (akcií). U akcií 1 až 5 se jedná o nedělitelné bloky (např. ucelené projekty), do kterých lze investovat buď plnou částku, nebo vůbec. Akcii číslo 6 lze nakupovat po jednotlivých kusech (nebo lotech).

Pro každou investici je známa pořizovací cena (5,7,4,3,4,6) a očekávaný výnos (16,22,12,8,11,19). Investor je omezen celkovým rozpočtem R

Při sestavování portfolia musí investor dodržet následující strategická pravidla:

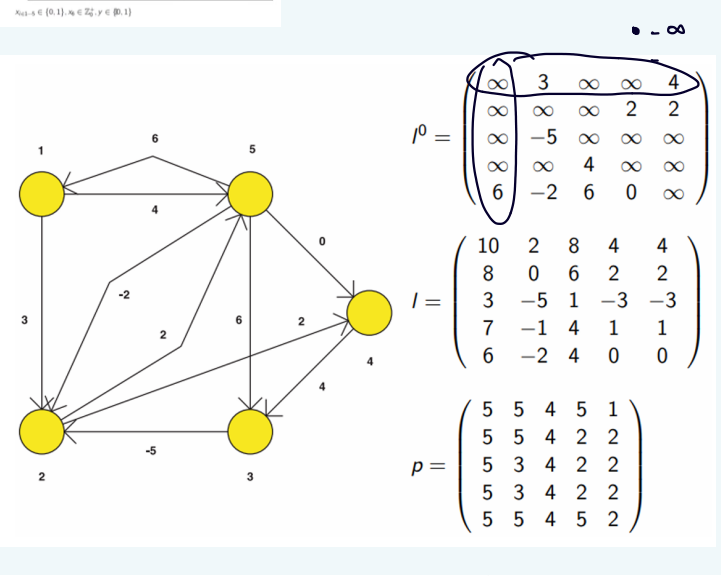
1. **Limit na počet kusů:** Od akcie 6 smí nakoupit maximálně 10 kusů.
2. **Podmíněná investice:** Pokud se investor rozhodne koupit akcii 2 *a zároveň* akcii 3, musí povinně do portfolia zařadit i akcii 4.
3. **Exkluzivita (XOR):** Investor musí zařadit do portfolia *právě jednu* z následujících dvou možností: buď koupí blok akcie 1, nebo nakoupí akcii 6 (v libovolném nenulovém množství). Nesmí vybrat obě tyto společnosti a nesmí je obě ignorovat.
4. **Diverzifikace:** Portfolio musí být diverzifikované. Investor musí investovat do **alespoň 3 různých společností**. (Poznámka: Nákup více kusů akcie 6 se pro účely diverzifikace počítá pouze jako jedna společnost).

Cíl: Formulujte model, který maximalizuje celkový očekávaný výnos při dodržení všech rozpočtových a logických omezení.



Stock Investment - solution

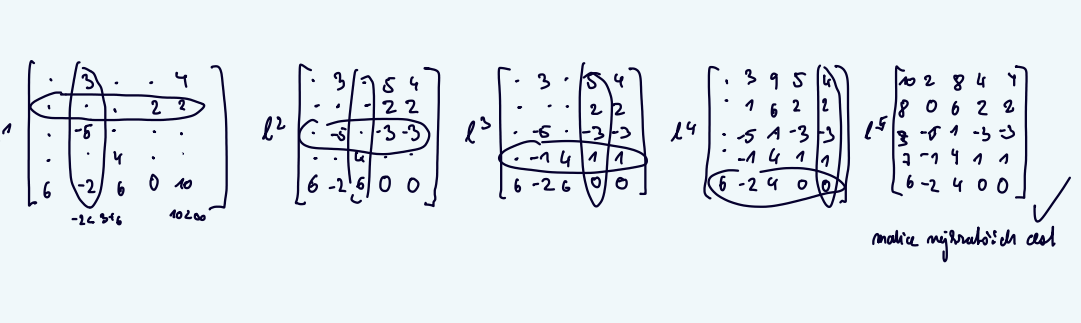
• − ∞



$$l^0 = \begin{pmatrix} \infty & 3 & \infty & \infty & 4 \\ \infty & \infty & \infty & 2 & 2 \\ \infty & -5 & \infty & \infty & \infty \\ \infty & \infty & 4 & \infty & \infty \\ 6 & -2 & 6 & 0 & \infty \end{pmatrix}$$

$$l = \begin{pmatrix} 10 & 2 & 8 & 4 & 4 \\ 8 & 0 & 6 & 2 & 2 \\ 3 & -5 & 1 & -3 & -3 \\ 7 & -1 & 4 & 1 & 1 \\ 6 & -2 & 4 & 0 & 0 \end{pmatrix}$$

$$p = \begin{pmatrix} 5 & 5 & 4 & 5 & 1 \\ 5 & 5 & 4 & 2 & 2 \\ 5 & 3 & 4 & 2 & 2 \\ 5 & 3 & 4 & 2 & 2 \\ 5 & 5 & 4 & 5 & 2 \end{pmatrix}$$



matice nejkratších cest

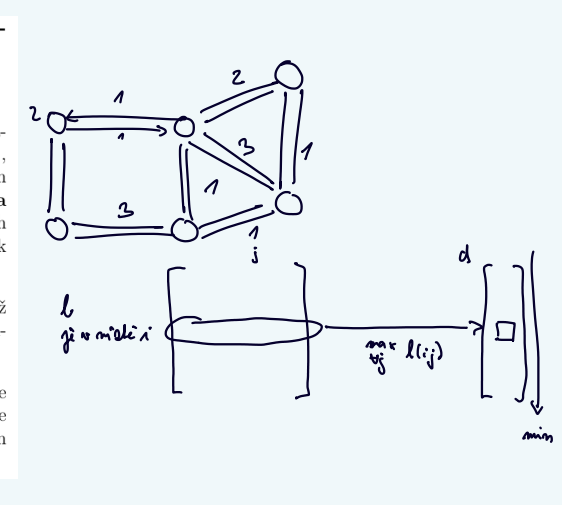
4.22.1 Příklad: Umístění požární stanice (Location of Fire Station)

Formulujte problém nejkratší cesty: Uvažujme silniční systém ve městě.

a) Hledáme nejlepší umístění požární stanice tak, aby byla minimalizována její vzdálenost od nejvzdálenějšího místa. To znamená, že hledáme uzel (umístění stanice), který minimalizuje maximum vzdáleností od tohoto uzlu ke všem ostatním uzlům v grafu. Tento problém je znám jako **minimax problém** nebo **problém centra** v grafu. Řešením by bylo pro každý možný uzel s vypočítat nejkratší cesty do všech ostatních uzlů t (pomocí SPT algoritmu), najít maximum z těchto vzdáleností, a pak vybrat s, pro které je toto maximum minimální.

b) **Úloha pro domácí cvičení:** Jak se problém změní (stane se složitějším), když hledáme nejlepší umístění pro **dvě požární stanice**? To vyžaduje pokročilejší techniky, pravděpodobně s ILP nebo speciálními algoritmy pro lokalizaci.

c) **Úloha pro domácí cvičení:** Jak se problém změní (stane se složitějším), když je dána **maximální povolená vzdálenost** (např. dojezdový čas 5 minut) a hledáme **minimální počet stanic**? To je známý **problém pokrytí**, který je obvykle řešen pomocí ILP nebo heuristik.



4.22.2 Příklad: Umístění skladu (Warehouse Location)

Formulujte problém nejkratší cesty: Uvažujme silniční systém ve městě, který zásobuje n zákazníků, z nichž každý spotřebovává $q_1, \dots, q_n$ jednotek zboží týdně. Je k dispozici vhodné místo pro sklad poblíž každého zákazníka. Každý zákazník je obsluhován samostatným vozidlem a náklady na přepravu jsou součinem vzdálenosti a přepravovaného objemu. Cílem je minimalizovat týdenní přepravní náklady.

Formulace jako problém nejkratší cesty (nebo spíše APSP):

- **Uzly grafu:** Uzly představují všechna možná místa pro sklad (např. poblíž každého zákazníka) a místa zákazníků.
- **Hrany grafu:** Hrany představují cesty mezi těmito uzly, s váhami odpovídajícími vzdálenostem (nebo časům).
- **Cíl:** Pro každé potenciální místo skladu L vypočítáme celkové týdenní přepravní náklady. Ty jsou dány součtem nákladů na doručení každému zákazníkovi j: $\sum_{j=1}^{n} (\text{vzdálenost}(L,j) \cdot q_j)$. Nej kratší vzdálenost z L do j se najde pomocí APSP algoritmu. Problém spočívá v nalezení takového místa skladu L, pro které je tato celková suma nákladů minimální.

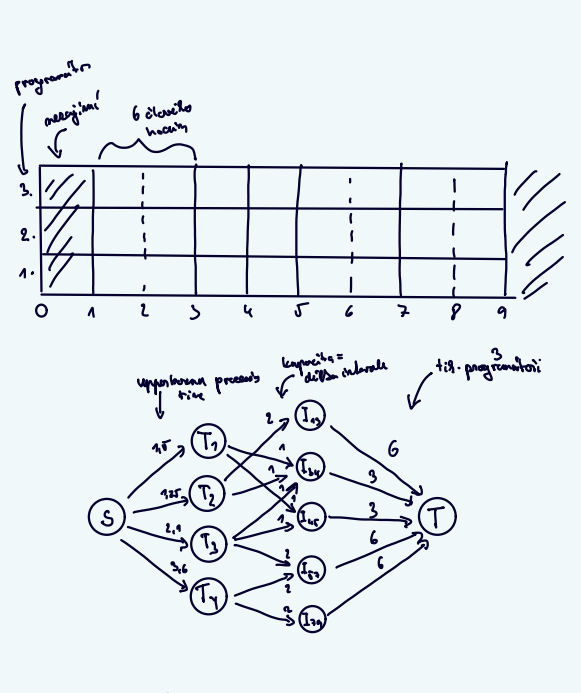
5.2.1 Příklad: Rozvrhování na multiprocesorech s preempcí

Uvažujeme problém $P \mid pmtn, r_j, \tilde{d}_j \mid -$. Máme n **úloh** (tasks), které chceme přiřadit R **identickým zdrojům** (procesorům). Každá úloha j má svou **dobu zpracování** p_j, **čas uvolnění** (release date) r_j a **termín dokončení** (deadline) $\tilde{d}_j$. **Preempce** (přerušení úlohy a její pozdější dokončení, včetně migrace na jiný zdroj) je povolena.

Příklad pro 3 paralelní identické zdroje:

úloha	T_1	T_2	T_3	T_4
p_j	1.5	1.25	2.1	3.6
r_j	3	1	3	5
$\tilde{d}_j$	5	4	7	9

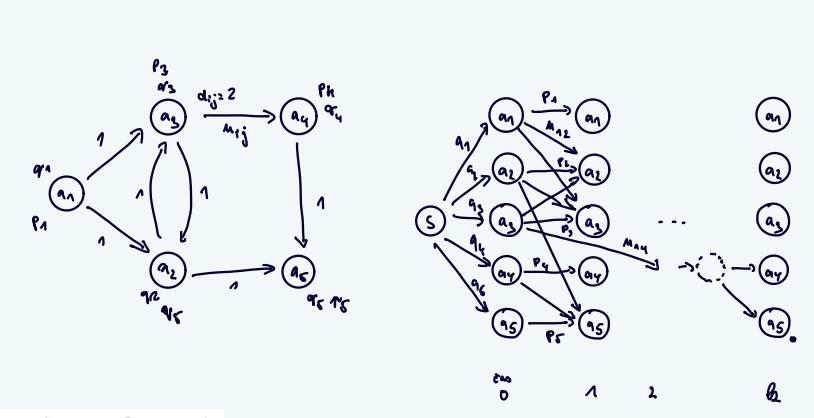
Cíl: Přiřadit všechny úlohy procesorům tak, aby každý procesor vykonával nanejvýš jednu úlohu v daném okamžiku a žádná úloha nebyla vykonávána současně na více než jednom procesoru.



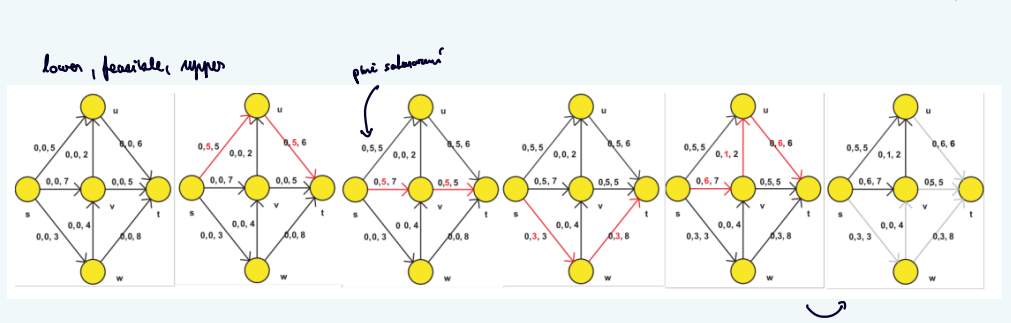
5.2.2 Příklad: Dynamický tok (Dynamic Flow)

Dynamický tok je tok, který mění svůj objem v čase. Problém s dynamickým tokem lze formulovat převedením na diskrétní čas a použitím (statického) toku na expandované síti.

Příklad: Uvažujme města $a_1, a_2, \dots, a_n$ s $q_1, q_2, \dots, q_n$ auty, která mají být přepravena do města a_n během K hodin. Když jsou města a_i, a_j spojena přímo, doba jízdy je označena d_{ij} a kapacita silnice (počet aut za hodinu) je u_{ij}. Parametr p_i reprezentuje kapacitu parkovací plochy ve městě a_i. Cílem je přepravit maximální počet aut do města a_n za K hodin.



lower, feasible, upper   plné nasycení



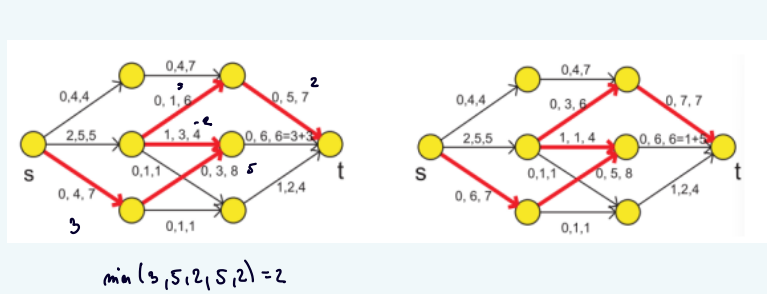
počáteční hodnoty

S → u A → t
kapacita 5 6
teče −0 −0
5 6
min → 5

$s = 5 + 6 + 3 = 14$ ✓
$t = 6 + 5 + 3 = 14$

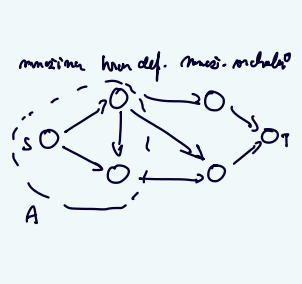
min. capacity cut $A = \{s, u, v\}$
po nasycených hranách

kapacita min řezu = max tok
hrany z množiny A: s → w $A = 3$
v → t $A = 5$
u → t $A = 6$ } → 14 ✓



min(3,5,2,5,2) = 2

maximum hran def. množ. vrcholů



$\delta^+(A)$ výstup
$\delta^-(A)$ vstup

Example Flow.cut.blocking: Blocking of Gasoline Supply

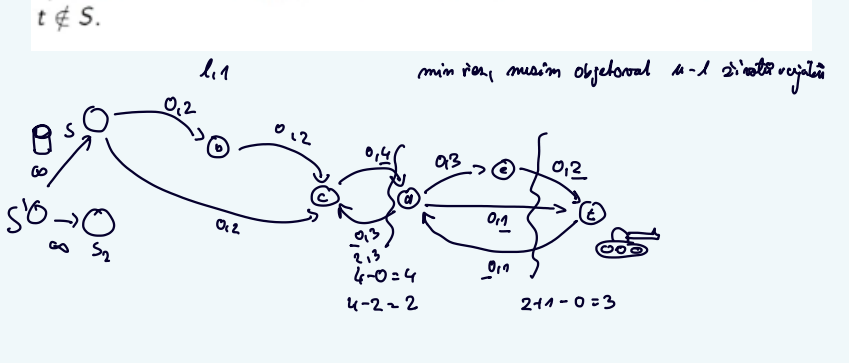
An adversary has a tank located at one node t and the gasoline supply pipeline denoted by directed graph $G(E,V)$. Gasoline supply node is denoted by s. Let pipe e has minimum throughput $l(e)$ and maximum throughput $u(e)$. We may decrease $u(e)$ by $k(e) \in \langle 0, u(e) - l(e) \rangle$, but in such case we loose $k(e)$ soldiers.

We want to isolate the tank from the gasoline supply while losing a minimum number of soldiers.

a) Consider $l(e) = 0$ and determine the minimal effort required to isolate the tank from a gasoline.

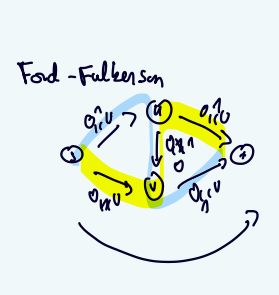
b) How the problem changes when we consider non-zero minimum throughput

c) Consider several supply nodes denoted by the set $S \subset V$ such that $t \notin S$.



min řez, musím obětovat u−l života vojáků

Ford-Fulkerson



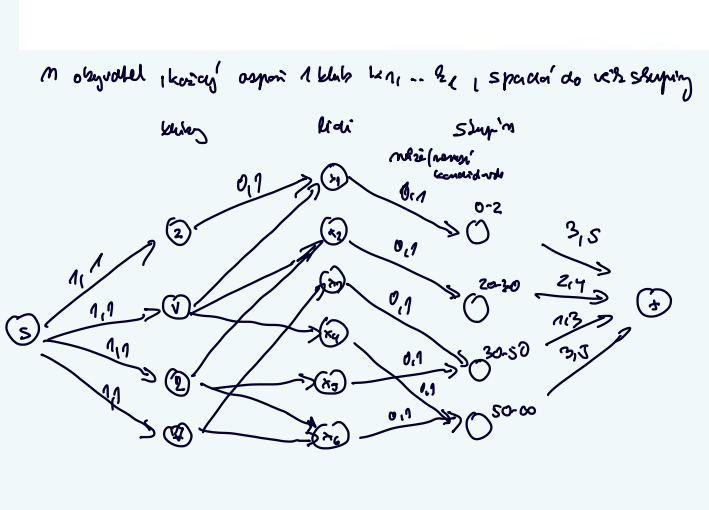
1. +1
2. +1
3(1.) +1
4(2.) +1

2U

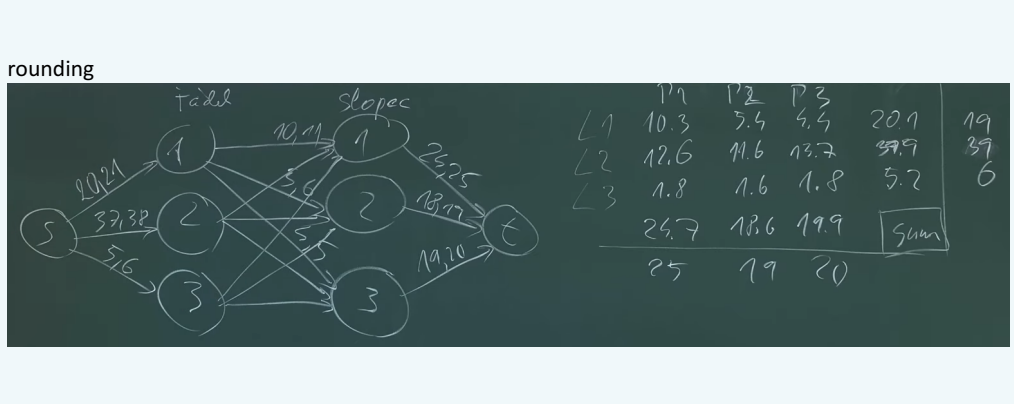
Example Flow.lower.representatives: Distinct Representatives [AMO93] - Integrality and Existence of Lower Bound

Assignment problem with an additional constraint.
Let us have n residents of a town, each of them is a member of at least one club $k_1, \dots, k_l$ and belongs to exactly one age group $p_1, \dots, p_r$. Each club must nominate one of its members to the town's governing council so that the number of council members belonging to the age group is constrained by its minimum and maximum. One resident represents at most one club. Find a feasible assignment of the representatives or prove that it does not exist. Formulate as the Maximum Flow Problem.

m obyvatel, každý aspoň 1 klub $k_1 \dots k_l$, spadají do věk. skupiny



rounding



2-Approximation Algorithm for Knapsack

r-approximation algorithm for maximization

Algorithm A for objective function J maximization is called r-approximation if there exists a number $r \ge 1$ such that $J^A(I) \ge \frac{1}{r} J^*(I)$ for all instances I of this problem.

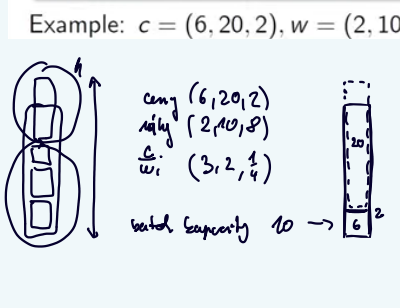
Theorem

Let $n, c_1, \dots, c_n, w_1, \dots, w_n, W, h$ are nonnegative integers that satisfy:

- $w_j \le W$ pro $j = 1, \dots, n$
- $\sum_{i=1}^{n} w_i > W$
- $\frac{c_1}{w_1} \ge \frac{c_2}{w_2} \ge \dots \ge \frac{c_n}{w_n}$
- $h = \min \left\{ j \in \{1, \dots, n\} : \sum_{i=1}^{j} w_i > W \right\}$

Then choosing the better of the two solutions $\{1, \dots, h-1\}$ or $\{h\}$ is **2-approximation alg. for the Knapsack** with the time complexity $O(n)$.

Example: $c = (6, 20, 2)$, $w = (2, 10, 8)$, $W = 10$.



ceny (6,20,2)
váhy (2,10,8)
$\frac{c_i}{w_i}$ (3,2,¼)
batoh kapacity 10 → 6
beru 10 → hodnota 20

Dynamic Programming (Integer Costs) for Knapsack

- Pseudopolynomial algorithm with time complexity $O(nC)$.
- Variable x_k^j represents the **minimum weight with cost k which can be achieved as a selection of items from set $\{1, \dots, j\}$**
- (*) Item j is added to the selection of items from $1, \dots, j$ if for the given price k this set reaches the **lower or equal weight as set** $1, \dots, j-1$.

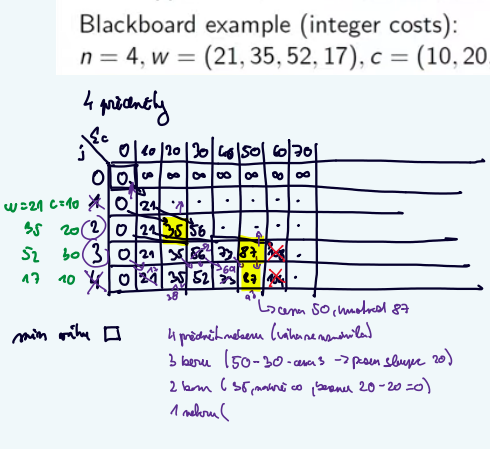
  The algorithm computes these values using the recursion formula:

$$x_k^j = \begin{cases} x_{k-c_j}^{j-1} + w_j & \text{if item } j \text{ was added;} \\ x_k^{j-1} & \text{if item } j \text{ wasn't added.} \end{cases}$$

- In variable s_k^j we memorize which of the two possible cases has happened. It is later used to reconstruct the selection.

Blackboard example (integer costs):
$n = 4$, $w = (21, 35, 52, 17)$, $c = (10, 20, 30, 10)$, $W = 100$.

4 předměty   kapacita


j
c
0
10
20
30
40
50
60
70
w=21 c=10
35 20
52 30
17 10
∞
21
35
56
87
→ cena 50, hmotnost 87

min větší □

4 předmět nebere (ukazuje nahoru)
3 beru (50−30 = cena 5 → poslední sloupec 20)
2 beru (35, ... (cena 20−20 = 0)
1 neberu

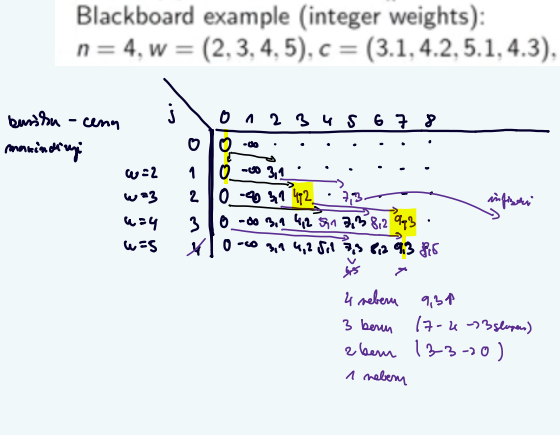
Dynamic programming for Knapsack - Overview

Dynamic programming overview

- Pseudopolynomial Algorithm
- State space (may be represented by a graph) is constructed due to **integer weights or costs** of items.
- Due to the **optimal substructure** property of the problem, the optimal solution may be found by the **recurrent formula** using **Bellman's Principle of Optimality**.
- The problem has **overlapping subproblems**. State space is not the tree, it contains **diamonds**, places in the state space where we keep only the better of two possible solutions - refer to item (*) - prevents exponential growth of the state space size.

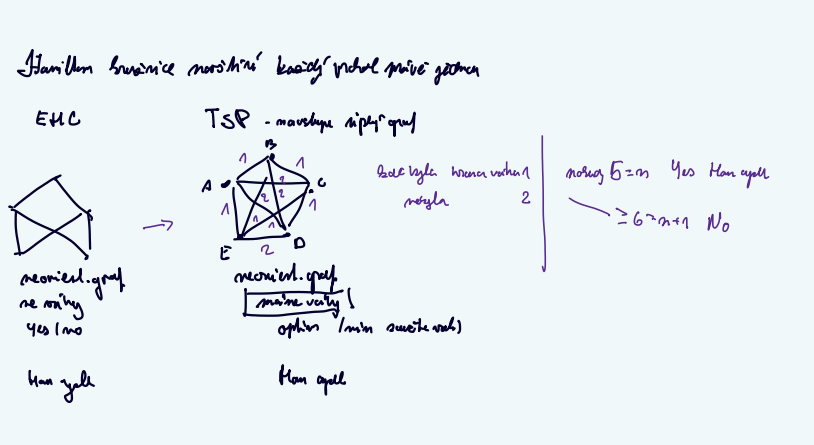
If the weights are integers, we can solve the problem by dynamic programming while selecting the solution having the **higher** cost for given weight (*) while initializing $x_k^0 := -\infty$ for $k = 1, \dots, C$.

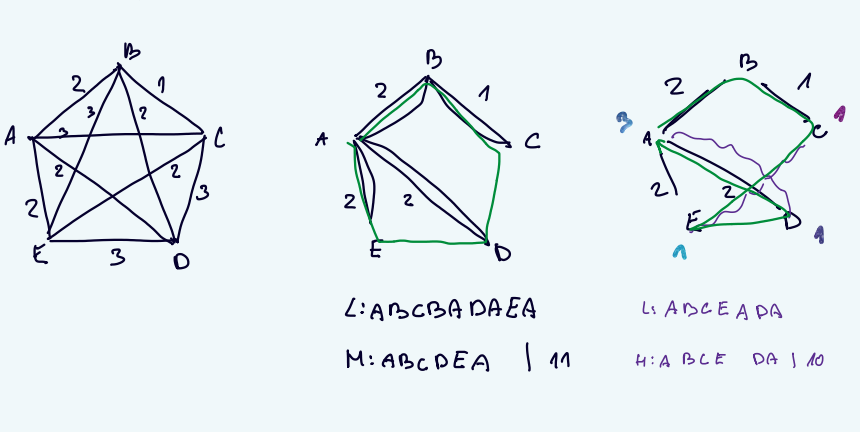
Blackboard example (integer weights):
$n = 4$, $w = (2, 3, 4, 5)$, $c = (3.1, 4.2, 5.1, 4.3)$, $W = 8$.


hmotnost - cena maximalizuji
j
0 1 2 3 4 5 6 7 8
w=2
w=3
w=4
w=5
0
1
2
3
4
−∞
3,1
4,2
7,3
8,3
9,3
8,5
stejné váhy
nižší

4 neberu 9,3 ✓
3 beru (7 − 4 → 3 sloupec)
2 beru (3 − 3 → 0)
1 neberu

Hamilton. kružnice navštíví každý vrchol právě jednou



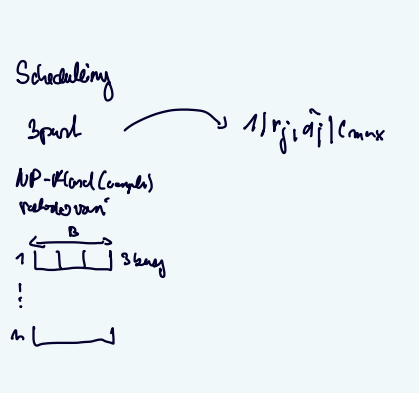


L: ABCBADAEA

M: ABCDEA | 11

L: ABCEADA

H: ABCE DA | 10



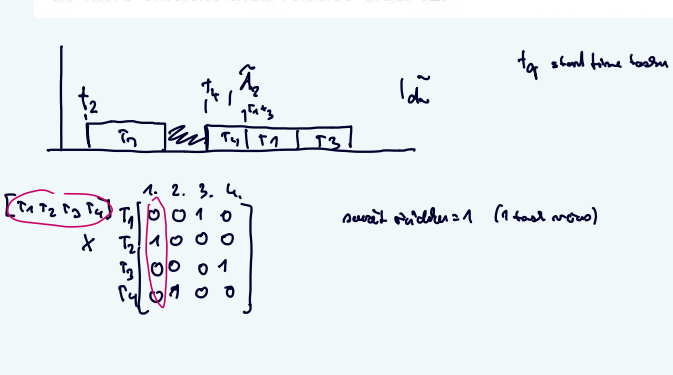
Position based ILP formulation for $1|r_j, \tilde{d}_j|C_{max}$

$x_{iq} = 1$ iff task i is at the q-th position in the sequence of tasks
t_q start time of task on the q-th position

$$\begin{aligned} &\min C_{max} \\ &\text{subject to:} \\ &\textstyle\sum_{q=1}^{n} x_{iq} = 1 && i = 1..n \\ &\textstyle\sum_{i=1}^{n} x_{iq} = 1 && q = 1..n \\ &t_q \ge \textstyle\sum_{i=1}^{n} r_i \cdot x_{iq} && q = 1..n \\ &t_q \ge t_{q-1} + \textstyle\sum_{i=1}^{n} p_i \cdot x_{i,q-1} && q = 2..n \\ &t_q \le \textstyle\sum_{i=1}^{n} \tilde{d}_i \cdot x_{i,q} - \textstyle\sum_{i=1}^{n} p_i \cdot x_{i,q} && q = 1..n \\ &C_{max} \ge t_n + \textstyle\sum_{i=1}^{n} p_i \cdot x_{in} \end{aligned}$$

variables: $x_{i \in 1..n, q \in 1..n} \in \{0,1\}$, $C_{max} \in \langle 0, UB \rangle$, $t_{q \in 1..n} \in \langle 0, UB \rangle$

… more efficient than relative order ILP

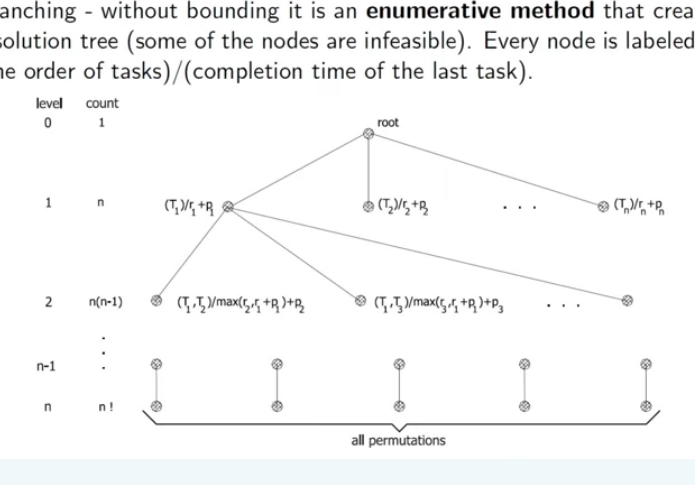


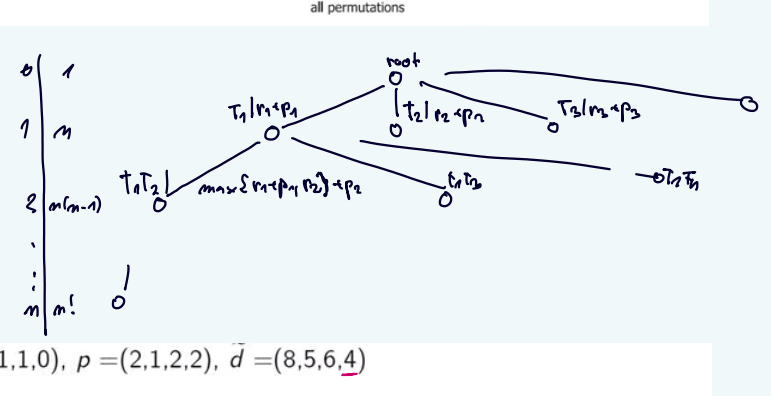
t_q start time tasku na q pozici

součet sloupce = 1 (1 task nanovo)

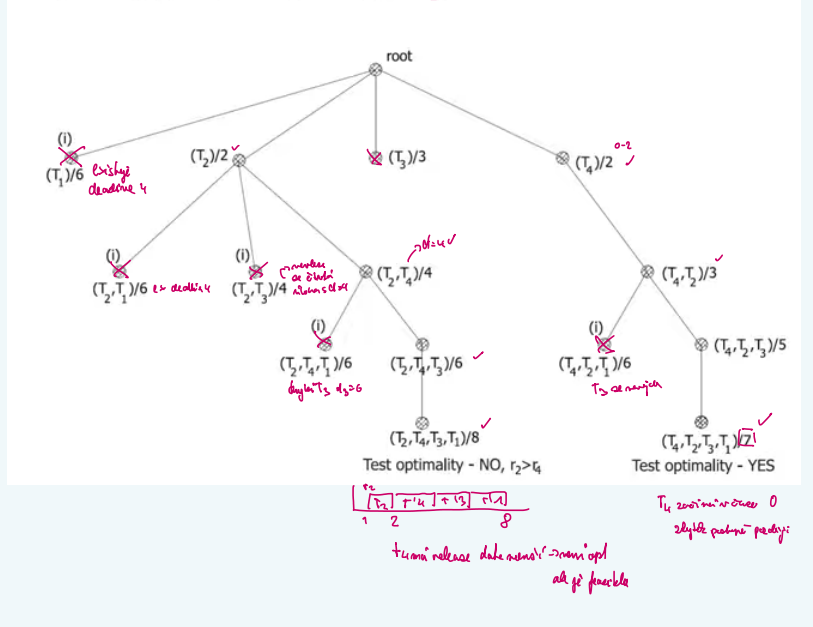
Bratley's Algorithm for $1|r_j, \tilde{d}_j|C_{max}$

A **branch and bound** (B&B) algorithm.
Branching - without bounding it is an **enumerative method** that creates a solution tree (some of the nodes are infeasible). Every node is labeled by: (the order of tasks)/(completion time of the last task).





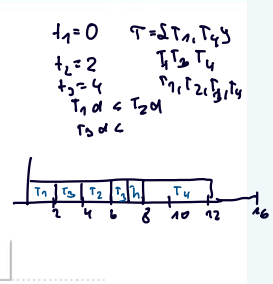
$r = (4,1,1,0)$, $p = (2,1,2,2)$, $\tilde{d} = (8,5,6,4)$

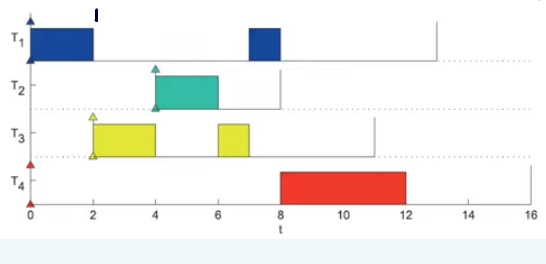


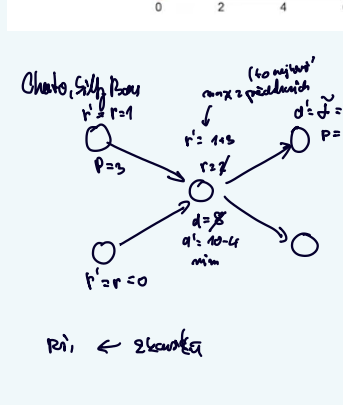
Problem $1|pmtn, r_j|L_{max}$

Example:

$$\begin{aligned} \mathcal{T} &= \{T_1, T_2, T_3, T_4\} \\ p &= (3, 2, 3, 4) \\ r &= (0, 4, 2, 8) \\ d &= (13, 8, 11, 16) \end{aligned}$$







r_i ← zbourka

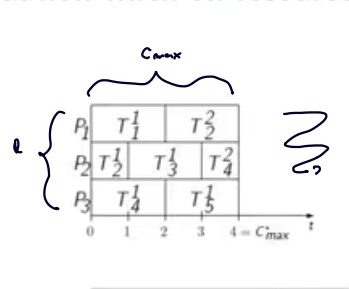
McNaughtnon's Algorithm for $P|pmtn|C_{max}$

The term $C^*_{max} = \max \left\{ \max_{i=1 \dots n} \{p_i\}, \frac{1}{R} \sum_1^n p_i \right\}$ should be interpreted as follows:

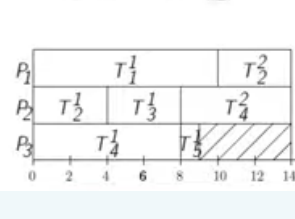
- component $\max_{i=1 \dots n} \{p_i\}$ represents the sequential nature of each task - it's parts can be assigned to different resources, but these parts can not be run simultaneously. Note that each task can be divided into two parts at most.
- component $\frac{1}{R} \sum_1^n p_i$ represents a situation when all resources work without idle waiting

Example 1:
$p = (2, 3, 2, 3, 2)$, $R = 3$
compute $C^*_{max} = \max \left\{ 3, \frac{12}{3} \right\} = 4$

nejdelší p $\frac{2+3+2+3+2}{R=3} = 4$

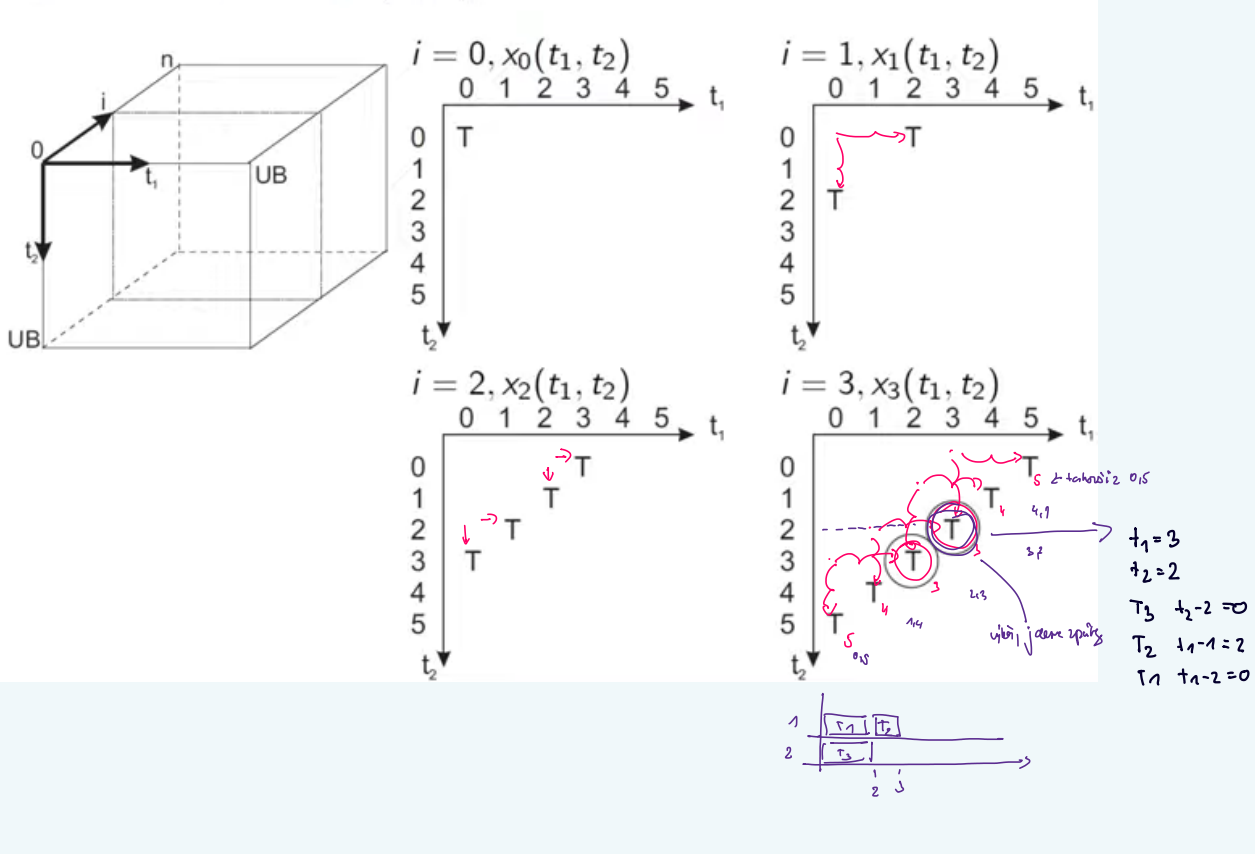


Example 2:
$p = (10, 8, 4, 14, 1)$, $R = 3$
compute $C^*_{max} = \max \left\{ 14, \frac{37}{3} \right\} = 14$

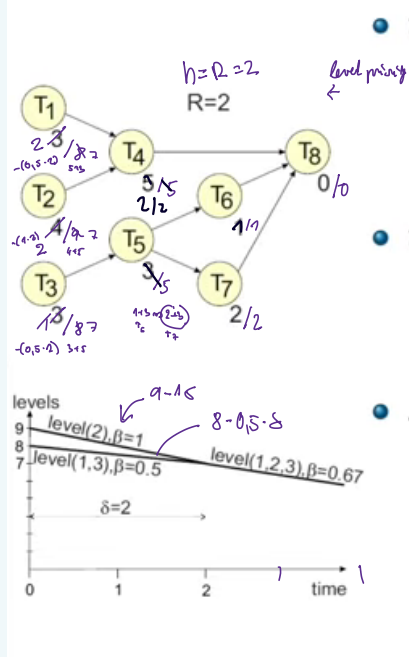


Example for $P||C_{max}$ [Rothkopf]

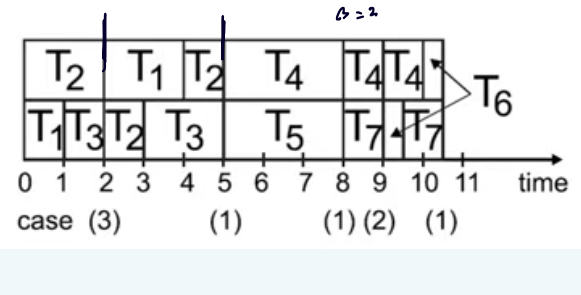
Example n=3, R=2, p=(2,1,2), UB=5.



Example - Muntz&Coffman's Alg. for $P|pmtn, prec|C_{max}$



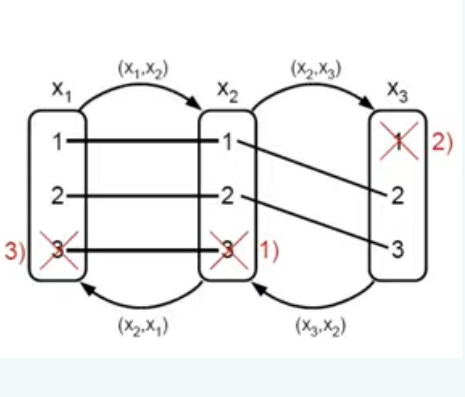
- $t = 0$, $p = (3, 4, 3, 5, 3, 1, 2, 0)$
  $level = (8, 9, 8, 5, 5, 1, 2, 0)$, $\mathcal{Z} = \{T_1, T_2, T_3\}$
    - $h = 2$, $\mathcal{S} = \{T_2\}$, $\beta = 1$ $\beta = \frac{h}{|S|}$
    - $h = 1$, $\mathcal{S} = \{T_1, T_3\}$, $\beta = 0.5 = \frac{1}{2}$
    - $\delta = 2$ due to the case (3) $9 - 1\delta = 8 - 0{,}5\delta$, $1 = 0{,}5\delta$ → $\delta = 2$
- $t = 2$, $p = (2, 2, 2, 5, 3, 1, 2, 0)$
  $level = (7, 7, 7, 5, 5, 1, 2, 0)$, $\mathcal{Z} = \{T_1, T_2, T_3\}$
    - $h = 2$, $\mathcal{S} = \{T_1, T_2, T_3\}$, $\beta = 0.67 = \frac{2}{3}$
    - $\delta = 3$ due to the case (1) $\delta = \frac{\text{práce}}{\text{rychlost}} = \frac{2}{\frac{2}{3}} = 3$
- $t = 5$ $h = 2$, $\mathcal{S} = \{T_4, T_5\}$ $\beta = 1$ $\delta = \frac{3}{1} = 3$
  $Z = \{T_4, T_5, T_7\}$ $S = \{T_4, T_5\}$ $\beta = 1$

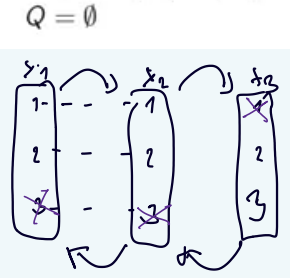


Example: Iteration of AC-3

CSP with variables and dom. $x_1 \in \{1,2,3\}, x_2 \in \{1,2,3\}, x_3 \in \{1,2,3\}$ and constraints $x_1 = x_2$, $x_2 + 1 = x_3$.

Queue initialization: $Q = ((x_1,x_2),(x_2,x_1),(x_2,x_3),(x_3,x_2))$
revise (x_1,x_2)
$x_1 \in \{1,2,3\}, x_2 \in \{1,2,3\}, x_3 \in \{1,2,3\}$
$Q = \{(x_2,x_1),(x_2,x_3),(x_3,x_2)\}$
revise (x_2,x_1)
$x_1 \in \{1,2,3\}, x_2 \in \{1,2,3\}, x_3 \in \{1,2,3\}$
$Q = \{(x_2,x_3),(x_3,x_2)\}$
revise (x_2,x_3)
$x_1 \in \{1,2,3\}, x_2 \in \{1,2\}^{1)}, x_3 = \{1,2,3\}$
$Q = \{(x_3,x_2),(\mathbf{x_1},\mathbf{x_2})\}$
revise (x_3,x_2)
$x_1 \in \{1,2,3\}, x_2 \in \{1,2\}, x_3 \in \{2,3\}^{2)}$
$Q = \{(x_1,x_2)\}$
revise (x_1,x_2)
$x_1 \in \{1,2\}^{3)}, x_2 \in \{1,2\}, x_3 \in \{2,3\}$
$Q = \emptyset$





Q
~~(x_1,x_2)~~
~~(x_2,x_1)~~
~~(x_2,x_3)~~
~~(x_3,x_2)~~
~~(x_1,x_2)~~

$3+1 = 4 \notin x$ smažu v x_2, musím přidat $(\dots, x_2)$
$1+1 = 2$ nejde 1 z x_3
x_2 změní x_1 smazán